

ПОЈМОВНИК ЗА ОДБРАНУ

Објашњење и пример за сваки термин коришћен у раду

А. Основни појмови

Лог / logfajl

Текстуални запис о томе шта се десило у програму током рада. Логови се чувају у фајловима (logfajlovima) или шаљу на централно место ради прегледа и претраге.

Пример: *Кад корисник клика „Пријави се“, апликација записује лог: „AuthService: корисник marko@mail.com успешно пријављен“.*

Обсервабилност (observability)

Способност да се, на основу спољашњих података које систем емитује (логови, метрике, trace-ови), закључи шта се дешава УНУТАР система — без потребе да се ручно „гађа“ и проверава сваки део кода посебно.

Пример: *Ако корисник каже „апликација ми ради споро“, уз обсервабилност отвориш Grafana и тачно видиш КОЈИ сервис и КОЈИ SQL упит је узрок, без нагађања.*

Health check

Механизам којим сервис сам „пријављује“ да ли је жив и да ли ради исправно, обично кроз посебан URL (endpoint) који неко (човек или алат) може да позове.

Пример: */health/ready на TripService-у враћа „Healthy“ ако је база доступна, или „Unhealthy“ ако није.*

Минимално логовање

Стање у ком код готово ништа не записује о свом раду, па је немогуће касније утврдити шта се тачно догодило кад нешто крене наопако.

Пример: *Пре овог рада, цео backend је имао 0 позива ILogger — ниједан део кода није писао ништа у лог.*

„Тихе“ грешке

Грешке које се десе у коду (нпр. мрежни позив не успе), али се НИГДЕ не забележе — програм их „прогута“ и настави даље као да се ништа није десило.

Пример: *Кад се брише кориснички налог, AuthService позива TripService да обрише његове планове; ако тај позив не успе, код је само враћао false без иједног лога — нико никад не би сазнао да су планови остали у бази.*

ILogger

Уграђен алат (интерфејс) у .NET-у који програмери користе да би у коду записали log поруку.

Пример: *_logger.LogWarning("TripService је одбио брисање планова корисника X") — једна линија кода која креира log запис.*

Б. Метрике и перформансе

Метрика перформанси

Број који описује КОЛИКО ДОБРО систем ради у датом тренутку — нпр. колико захтева у секунди, колико је систем спор, колико грешака прави.

Пример: „TripService тренутно прима 12 захтева у секунди“ је једна метрика.

CPU

Процесор — „мозак“ рачунара који извршава код. Метрика „CPU искоришћеност“ показује колико је процесор заузет, изражено у процентима.

Пример: Ако CPU искоришћеност скочи на 95%, то значи да је сервис близу границе својих могућности.

Латенција

Време које протекне од тренутка кад је захтев послат до тренутка кад је стигао одговор. Краће = брже.

Пример: Ако кликнеш „учитај планове“ и одговор стигне за 200 милисекунди, латенција тог захтева је 200ms.

Стопа грешака (error rate)

Проценат захтева који су завршили грешком, од укупног броја захтева.

Пример: Ако од 100 захтева 5 врати грешку, стопа грешака је 5%.

p95 (латенција)

Вредност латенције испод које спада 95% свих захтева — другим речима, само најспоријих 5% захтева је спорије од ове бројке. Користи се уместо просека, јер просек сакрива „лоше“ случајеве.

Пример: p95 латенција од 2.14 секунде значи да је 95% захтева било брже од 2.14s, а најспоријих 5% је било спорије од тога.

В. Дистрибуирано праћење захтева (tracing)

Correlation ID / Trace ID

Јединствени број/код који се додели ЈЕДНОМ корисничком захтеву на самом почетку, и тај исти код „путује“ кроз СВЕ сервисе кроз које тај захтев пролази. Тако се касније могу повезати сви логови и акције које припадају тачно том једном захтеву, чак и кад су настале у различитим сервисима.

Пример: Захтев „прикажи ми план X“ добије trace ID „abc123“ на Gateway-у; тај исти „abc123“ се преноси до SharingService-а и TripService-а, па касније можеш да претражиш све логове са „abc123“ и видиш ЦЕО пут тог захтева.

Span (нпр. „9 SQL child span-ова“)

Један trace се састоји од више мањих делова званих span-ови — сваки span представља ЈЕДНУ операцију (нпр. један HTTP позив, или један SQL упит). „Child span“ значи да је та операција део неке веће операције (нпр. SQL упит унутар HTTP захтева).

***Пример:** Кад TripService учита цео план путовања, он чита из 9 различитих табела у бази — сваки од тих 9 SQL упита добија свој span, „угнежђен“ (као дете) унутар главног span-а тог захтева. Кад отвориш trace у Тетро-у, видиш свих 9 као мање траке испод главне.*

Log → Trace → Metric (навигација)

У Grafani, кад видиш једну линију лога, можеш једним кликом да одеш на trace коме та линија припада, а из trace-а једним кликом на повезану метрику — све је „повезано“.

***Пример:** Видиш у Loki-ју log „грешка у TripService-у“ → кликнеш и отвара ти се цео trace тог захтева у Тетро-у → видиш тачно који SQL упит је био узрок.*

Г. OpenTelemetry и стандард

OpenTelemetry (OTel)

Бесплатан, опште прихваћен стандард (скуп правила и алата) за прикупљање три врсте података о систему — trace-ове, метрике и логове — на ЈЕДАН, јединствен начин, без обзира на то који програмски језик или алат користиш. „OTel“ је само скраћеница за OpenTelemetry.

***Пример:** Уместо да сама пишеш код који шаље податке у Prometheus, па други код за Тетро, па трећи за Loki — користиш OpenTelemetry као један заједнички „језик“ за све троје.*

Vendor-неутралан

Значи „независан од конкретног произвођача/алата“. Ако је нешто vendor-неутрално, можеш касније да промениш алат (нпр. Prometheus у нешто друго) БЕЗ да мораш да мењаш код апликације.

***Пример:** Зато што је OpenTelemetry vendor-неутралан, сутра можеш да замениш Grafana Loki неким другим алатом за логове, а да не дираш ниједан сервис — само промениш подешавања Collector-а.*

„Заједнички API/SDK за traces, metrics, logs — auto-инструментација за ASP.NET Core, HttpClient, SqlConnection“

API/SDK је скуп готових алата/библиотека које програмер укључи у свој код да би нешто урадио (овде: да би слао телеметрију). „Auto-инструментација“ значи да OpenTelemetry АУТОМАТСКИ прати одређене делове кода БЕЗ да програмер мора ручно да пише код за то. ASP.NET Core је технологија на којој су направљени backend сервиси (Gateway, AuthService, итд.). HttpClient је део кода који се користи кад један сервис позива други преко мреже. SqlConnection је део кода који се користи кад сервис комуницира са SQL базом.

***Пример:** Цела реченица значи: OpenTelemetry има готов, заједнички алат који АУТОМАТСКИ прати све HTTP позиве, позиве ка бази и основне операције у ASP.NET Core апликацијама — а да програмер не мора ручно да пише код за праћење сваког позива појединачно.*

„Централна тачка прикупљања телеметрије (OTLP, gRPC) — раздваја емисију од складиштења“

„Централна тачка“ је OpenTelemetry Collector, посебан програм коме СВИ сервиси шаљу своје податке. OTLP је протокол (договорени „језик“/формат) којим се телеметрија шаље ка Collector-у. gRPC је технички начин/канал којим се ти подаци физички преносе преко мреже. „Раздваја емисију од складиштења“ значи да сервиси не морају да знају ГДЕ се подаци на крају чувају — они само шаљу све ка Collector-у, а Collector одлучује где ће шта да проследи.

Пример: Уместо да сваки сервис директно шаље податке у Prometheus/Tempo/Loki, сви шаљу само ка једном централном програму (Collector-у, преко OTLP/gRPC канала) — а тај програм даље распоређује податке где треба. Тако сервиси не зависе од тога ГДЕ се подаци на крају складиште.

Д. Алати за складиштење и визуелизацију

Prometheus

Алат/база података специјализован за чување и претрагу МЕТРИКА — бројева који се мењају кроз време (нпр. број захтева у секунди).

Пример: Prometheus сваких пар секунди „оде“ и прочита тренутне бројке са сваког сервиса (тај процес се зове „scrape“), и чува историју тих бројки.

Grafana

Алат за ВИЗУЕЛИЗАЦИЈУ — прави графиконе, dashboard-е и омогућава претрагу података из Prometheus-а, Tempo-а и Loki-ја, све на једном месту, кроз визуелни интерфејс.

Пример: Dashboard „Travel Planner - Overview“ приказује графиконе за све сервисе одједном.

Grafana Tempo

Алат специјализован за чување TRACE-OBA — путања појединачних захтева кроз систем.

Пример: Кад желиш да видиш тачно којим путем је прошао ЈЕДАН конкретан захтев, отвориш га у Tempo-у.

Grafana Loki

Алат специјализован за чување ЛОГОВА из свих сервиса на једном месту, са могућношћу претраге.

Пример: Уместо да отвараш 4 различита фајла са логовима за 4 сервиса, у Loki-ју их све претражујеш заједно.

Ђ. Тестирање и инфраструктура

xUnit

Алат (framework) за писање аутоматизованих тестова у .NET-у — помоћу њега се пише код који сам проверава да ли остатак кода ради исправно.

Пример: Тест „DeleteUserTripsAsync_WhenTripServiceReturnsError_ReturnsFalseAndLogsWarning“ проверава да ли код исправно реагује кад TripService врати грешку.

Kestrel host

Уграђен web-сервер у .NET-у који „слуша“ на одређеном порту и прима HTTP захтеве. Кад тестови „подигну прав Kestrel host“, то значи да се покреће СТВАРНА верзија сервиса (не лажна/симулирана), само на привременом (ефемерном) порту, само за време трајања теста.

Пример: Тест покрене прав AuthService на порту нпр. 54321 само док тест траје, пошаље му прав HTTP захтев, и провери одговор.

Docker Compose

Алат који омогућава да се једном командом покрене ВИШЕ повезаних програма/контејнера одједном (нпр. Collector, Prometheus, Tempo, Loki, Grafana), уместо да се сваки покреће ручно.

***Пример:** Цела observability инфраструктура (5 алата) покреће се једном командом „docker compose up“, захваљујући једном конфигурационом фајлу.*

k6

Алат за тестирање ОПТЕРЕЂЕЊА (load testing) — симулира много корисника који истовремено користе апликацију, да би се видело како се систем понаша под притиском.

***Пример:** k6 тест је симулирао до 30 корисника који истовремено користе апликацију и послао укупно 780 захтева.*

CI/CD pipeline

Аутоматизован процес који се покреће сваки пут кад програмер промени код — аутоматски гради (build), тестира и (опционо) поставља (deploy) апликацију, без ручног рада.

***Пример:** Кад би k6 тест био део CI/CD pipeline-а, аутоматски би се покренуо при свакој новој верзији кода, да се одмах види да ли је нова верзија успорила систем.*